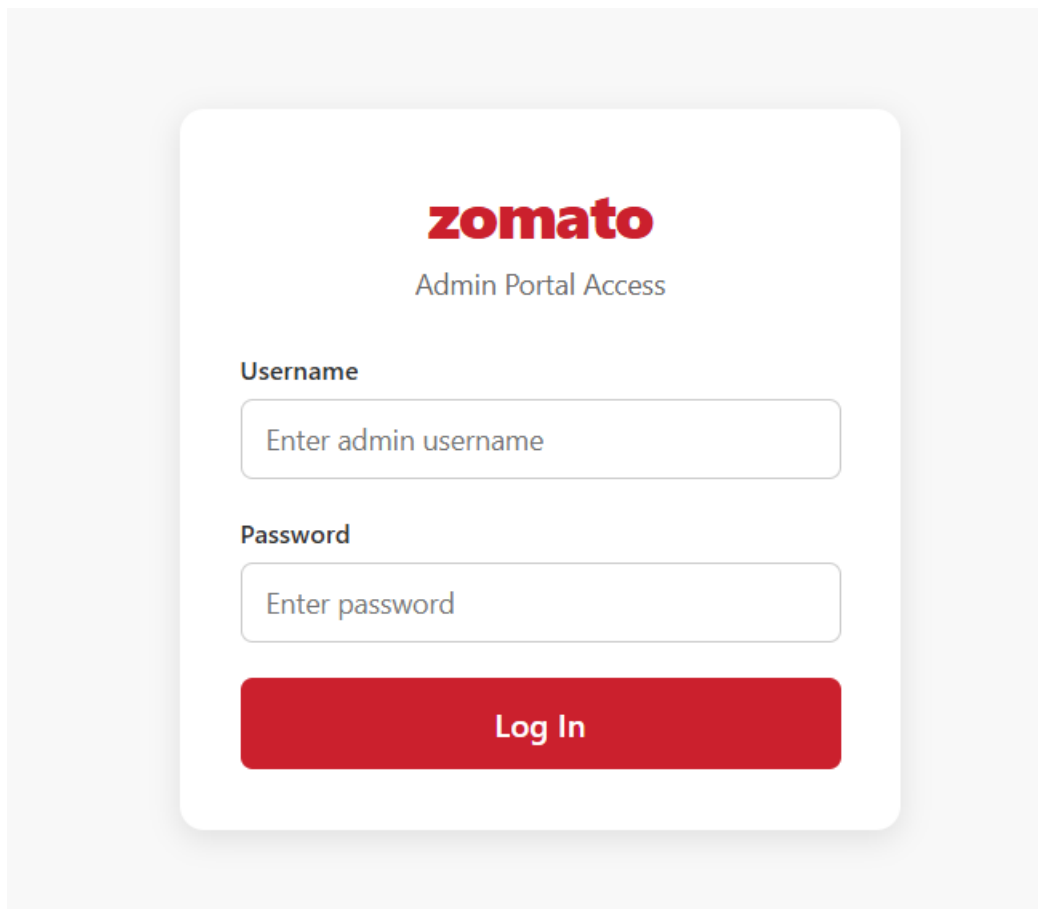


TASK-1



The image shows a login form for the Zomato Admin Portal. It features the Zomato logo at the top, followed by the text 'Admin Portal Access'. Below this are two input fields: one for 'Username' with the placeholder text 'Enter admin username', and one for 'Password' with the placeholder text 'Enter password'. At the bottom of the form is a red button labeled 'Log In'.

Credentials for Testing

- **Username:** admin
- **Password:** Zomato@2026

TASK-2

A Python function `check_access(user_role, resource)`

```
main.py x + Console I/O AI Agent 1/2 ms

1 def check_access(user_role: str, resource: str) -> bool:
2     """
3     Checks if a given user role is allowed to access a specific resource.
4
5     :param user_role: Role of the user ('customer', 'restaurant_owner', 'admin')
6     :param resource: Target resource ('order_history', 'menu_edit', 'admin_dashboard')
7     :return: True if access is permitted, False otherwise
8     """
9     # Dictionary mapping roles to a set of permitted resources
10    role_permissions = {
11        "customer": {
12            "view_menu",
13            "place_order",
14            "order_history"
15        },
16        "restaurant_owner": {
17            "view_menu",
18            "menu_edit",
19            "order_history",
20            "restaurant_analytics"
21        },
22        "admin": {
23            "view_menu",
24            "menu_edit",
25            "order_history",
26            "restaurant_analytics",
27            "admin_dashboard",
28            "user_management"
29        }
30    }
31
32    # Check if the user role is in the permissions dictionary
33    if user_role not in role_permissions:
34        return False
35
36    # Check if the resource is in the permitted resources for the user role
37    if resource not in role_permissions[user_role]:
38        return False
39
40    return True
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
--- Running Access Control Tests ---
Role: 'customer' | Resource: 'order_history' -> Access Granted: 1 |
Test: PASSED
Role: 'customer' | Resource: 'menu_edit' -> Access Granted: 0 |
Test: PASSED
Role: 'customer' | Resource: 'admin_dashboard' -> Access Granted: 0 |
Test: PASSED
Role: 'restaurant_owner' | Resource: 'menu_edit' -> Access Granted: 1 |
Test: PASSED
Role: 'restaurant_owner' | Resource: 'order_history' -> Access Granted: 1 |
Test: PASSED
Role: 'restaurant_owner' | Resource: 'admin_dashboard' -> Access Granted: 0 |
Test: PASSED
Role: 'admin' | Resource: 'admin_dashboard' -> Access Granted: 1 |
Test: PASSED
Role: 'admin' | Resource: 'menu_edit' -> Access Granted: 1 |
Test: PASSED
Role: 'guest' | Resource: 'order_history' -> Access Granted: 0 |
Test: PASSED
```

TASK-3

```
Console I/O AI Agent

=====
WELCOME TO PAYTM 2FA LOGIN
=====

Enter Username: admin
Enter Password: Paytm@1234

✅ Password verified successfully!

[SMS Gateway] Sending 6-digit OTP to your registered mobile number...

[📱 SMS ALERT]: Your Paytm Verification OTP is: 224044

Enter the 6-digit OTP (Attempt 1/3): 224044

=====
🎉 LOGIN SUCCESSFUL! Welcome to Paytm.
=====
```

TASK-4

JS index.js X +

Console I/O AI Agent 150 ms

```
1 function getActivePremiumUsers(users) {
2   // Fixed current date given in the constraint: June 10, 2024
3   const TODAY = new Date('2024-06-10');
4
5   // 7 days in milliseconds: 7 days * 24 hrs * 60 mins * 60 secs * 1000 ms
6   const SEVEN_DAYS_MS = 7 * 24 * 60 * 60 * 1000;
7
8   return users.filter(user => {
9     // Condition 1: Must be a premium user
10    if (!user.isPremium) return false;
11
12    // Convert the user's lastLogin string into a Date object
13    const loginDate = new Date(user.lastLogin);
14
15    // Calculate difference in time (in milliseconds)
16    const timeDiff = TODAY - loginDate;
17
18    // Condition 2: Logged in within 0 to 7 days (0 <= timeDiff <= 7 days)
19    return timeDiff >= 0 && timeDiff <= SEVEN_DAYS_MS;
20  });
21 }
```

```
[
  { username: 'rahl_07', isPremium: true, lastLogin: '2024-06-08' },
  { username: 'priya_v', isPremium: true, lastLogin: '2024-06-03' }
]
```

TASK – 5

Mistake	Description	Fix / Prevention Tip
1. Weak Password Policy	Users choose simple passwords that attackers can easily guess.	Enforce strong passwords and enable Multi-Factor Authentication (MFA).
2. Missing Role-Based Access Control (RBAC)	Regular users accidentally get access to admin features or sensitive data.	Assign permissions based on user roles (Admin, Seller, Customer, etc.).
3. Insecure Session Management	Login sessions remain active for too long or after logout, allowing unauthorized access.	Use secure session tokens and automatically expire inactive sessions.
4. Inadequate Authorization Checks	The application checks if a user is logged in but not whether they are allowed to access a specific resource.	Verify user permissions for every request before granting access.
5. Exposed APIs or Sensitive Data	APIs return confidential information without proper access validation.	Secure APIs with authentication, authorization, and data validation to restrict access.